

School Transport Management System

A part-by-part guide to the code

Object-Oriented Programming using Python | Group 6 Practical Project

Explains every class in `school_transport.py` in simple terms, using a school transport office as the running analogy, and answers the eleven assignment questions.

Presentation: Thursday 1st October 2026

1. The Big Picture

1.1 Classes and objects

Object-oriented programming (OOP) means building a program out of small, self-contained things that each know their own data and their own job, instead of one long list of instructions. Think of a school transport office. It does not have one worker doing everything. It has a manager, drivers, conductors, route maps and booking tickets, and each of them has a clear responsibility.

A **class** is the design or blueprint, like the manufacturer's specification for a minibus model. An **object** is one real thing built from that design, like the actual minibus parked in the school yard with its own number plate. You can build many objects from one class: one Van class can produce many vans, each with its own registration number and its own passengers.

1.2 The four big ideas

Encapsulation means protecting an object's data and letting the outside world reach it only through controlled doors. A vehicle's passenger compartment is locked; only the conductor decides who gets in.

Inheritance means a new class can reuse everything from an existing one. Buses, minibuses and vans are all vehicles, so they share the same basic vehicle features.

Polymorphism means the same instruction can produce different behaviour depending on the object receiving it. Ask any vehicle "what is your monthly fee?" and each answers in its own way. **Abstraction** means defining what something must be able to do without saying exactly how. The idea of "a school vehicle" is like that: you never park an idea in the yard, only a real bus, minibus or van.

1.3 The cast of characters

Here is every class in the program, matched to a person or object in our imaginary transport office.

Class	In the transport office	Its one job
Learner	A pupil's registration card	Holds and protects one learner's details
TransportVehicle (parent) with Bus, Minibus, Van	The vehicle design, and the real vehicles in the yard	Knows its seats, passengers, route and costs
Route	A road plan pinned on the notice board	Holds a name, stops, distance and the vehicles serving it
Subscription	The seat booking ticket	Records one learner on one vehicle on one route at one price
TransportSystem	The transport manager's office	Keeps the registers and carries out every transaction
TransportMenu	The reception desk	Talks to the user and passes the work to the office
TransportError, VehicleFullError	Warning signs and red flags	Announce that a rule has been broken

2. The File at a Glance and the Imports

The file is one long script organised in banner-marked sections, and it must be read from top to bottom because Python needs a class to be defined before another class uses it. The order is: exceptions, Learner, the vehicles, Route, Subscription, TransportSystem, the sample data, and finally the menu. The last two lines start the program.

```
import re
from abc import ABC, abstractmethod
```

Both lines come from Python's built-in standard library, so there is nothing to install. `re` gives us regular expressions, which are patterns for checking that text has the right shape, such as a phone number or a number plate. `ABC` and `abstractmethod` are the tools for abstraction: they let us design a blueprint that cannot be turned into a real object and that forces its children to fill in certain methods.

Transport analogy. *`re` is the checklist held by the gate guard: "a valid plate has 5 to 10 letters and digits". `ABC` is the stamp on a design document that says "this is a design only, you cannot drive it, but every vehicle built from it must follow it".*

3. The Exceptions: Traffic Warning Signs

```
class TransportError(Exception):
    """A business rule of the system was broken."""

class VehicleFullError(TransportError):
    """The vehicle has no free seats."""
```

An **exception** is Python's way of shouting "stop, something is wrong" and immediately abandoning the current operation. Python already has a class called `Exception`. `TransportError` inherits from it (that is what the brackets mean), so it becomes our own kind of error for broken transport rules. `VehicleFullError` then inherits from `TransportError`, making it a more specific version. This is the first inheritance chain in the file. The classes contain no working code because the docstring alone is a complete class body; their whole value is their name and their family tree.

The program uses two families of errors on purpose. The built-in `ValueError` is raised when a single piece of data is malformed, such as a phone number with letters in it. `TransportError` is raised when an operation breaks a rule, such as boarding a full vehicle. The menu catches both and prints a friendly message instead of crashing, and that is how requirement 10 (clear messages for invalid operations) is met.

Transport analogy. *`ValueError` is the gate guard rejecting a forged pupil ID card. `VehicleFullError` is the conductor holding up a hand and saying "the bus is full, you cannot board".*

4. The Learner Class: The Registration Card

A learner is the passenger in our story. The class stores three things about him or her and refuses to accept bad data.

```
def __init__(self, learner_id, name, grade, contact):
    self._learner_id = learner_id
    self.name = name
    self.contact = contact
    if not str(grade).strip():
        raise ValueError("Grade/class cannot be empty.")
    self.grade = str(grade).strip().upper()
```

The constructor

`__init__` is the **constructor**. Python runs it automatically the moment someone writes `Learner(...)`, and its job is to set the new object up. `self` means "this particular learner being built". So `self._learner_id = learner_id` writes the ID onto this learner's own card.

Transport analogy. The constructor is the moment a new pupil arrives at the gate and the clerk fills in their registration card. `self` is "this card in front of me", not somebody else's.

Public, protected and private

Notice the three styles of attribute name. A name with **no underscore**, such as `self.grade`, is **public**: anyone may read or change it. A name with **one underscore**, such as `self._learner_id`, is **protected**: a convention meaning "internal, please do not touch from outside". A name with **two underscores** is **private**, and you will meet one in the vehicle class. This mix of levels is exactly what requirement 3 asks you to demonstrate.

Properties and setters

```
@property
def learner_id(self):
    return self._learner_id

@property
def name(self):
    return self._name

@name.setter
def name(self, value):
    value = " ".join(str(value).split())
    if len(value) < 2 or not all(c.isalpha() or c in " -'." for c in value):
        raise ValueError("Name must have at least 2 characters and contain letters only.")
    self._name = value.title()
```

`@property` turns a method into something that looks like a plain attribute, so outside code writes `Learner.name` with no brackets. It is the **getter**. The matching `@name.setter` runs whenever someone writes `learner.name = "..."`, and that is where validation happens. The setter first tidies the text: `split()` breaks it at every space and `" ".join(...)` glues it back with single spaces, removing accidental doubles. It then rejects any name shorter than two characters or containing anything except letters, spaces and a few punctuation marks. If the name passes, `.title()` capitalises it and it is stored in the protected `_name`.

Because the constructor writes `self.name = name` (without the underscore), it goes through the setter, so it is impossible to create a learner with an invalid name. `learner_id` has a getter but no setter, so it is **read-only**: everyone can see the ID and nobody can change it. The contact property follows the same pattern with a regular expression.

```
if not re.fullmatch(r"0\d{9}|\+256\d{9}", value):
    raise ValueError("Guardian contact must be 10 digits starting with 0 (or +256 and 9 digits).")
```

Read the pattern piece by piece: `0\d{9}` means a 0 followed by exactly nine digits, the vertical bar means "or", and `\+256\d{9}` means +256 followed by nine digits. `fullmatch` demands that the entire text fits the pattern, not just a part of it.

Finally, `__str__` decides what appears when a learner is printed. Without it Python would print something unreadable like an object's memory address.

Transport analogy. The setter is the gate guard. He does not let anyone scribble on the registration card directly. Every change must pass his checks first. The read-only ID is the card number printed at the top: everyone can read it, no one may alter it.

5. The TransportVehicle Class: The Vehicle Blueprint

This is the most important class in the project. It carries abstraction, encapsulation and the top of the inheritance family all at once.

```
class TransportVehicle(ABC):
    TYPE_NAME = "Vehicle"
    MIN_CAPACITY, MAX_CAPACITY = 1, 100
    SCHOOL_DAYS = 22
```

(ABC) makes this an **abstract class**: Python will refuse to create a `TransportVehicle` directly, because it is only a blueprint for the classes beneath it. The three lines that follow are **class attributes**. They are written directly inside the class instead of inside `__init__`, so they belong to the whole model rather than to each individual vehicle. They are default values that the children will replace.

***Transport analogy.** Class attributes are the manufacturer's standard specifications stamped on the model, such as "this model seats 31 to 70". They are not stored on each individual vehicle in the yard.*

The constructor and the private passenger list

```
def __init__(self, registration, capacity):
    self.__passengers = []
    self._route = None
    registration = str(registration).strip().upper()
    if not re.fullmatch(r"[A-Z0-9 ]{5,10}", registration):
        raise ValueError("Registration must be 5-10 letters/digits (e.g. UBA 123X).")
    self._registration = registration
    self.capacity = capacity
```

`self.__passengers` has two leading underscores, which makes it **private**. Python secretly renames it inside the class so that outside code cannot touch it by accident. `self._route` is protected and starts as `None`, meaning "not yet assigned to any route". The registration is cleaned to capitals, checked against the pattern (5 to 10 letters, digits or spaces), and stored. The last line calls the capacity setter, so seat counts are validated too. The passenger list is created first because other code may need it.

***Transport analogy.** The passenger list is the locked passenger compartment. Nobody can reach in and add or remove people. The only way in or out is through the conductor, who is played by the methods `board()` and `alight()`.*

The capacity setter and the read-only windows

```
@capacity.setter
def capacity(self, value):
    if not isinstance(value, int) or not self.MIN_CAPACITY <= value <= self.MAX_CAPACITY:
        raise ValueError(f"A {self.TYPE_NAME} must have {self.MIN_CAPACITY}-{self.MAX_CAPACITY} seats.")
    self._capacity = value
```

The setter checks that the value is a whole number and lies within the model's allowed range. Look closely at `self.MIN_CAPACITY`. When the object is a `Bus` this reads the `Bus` limits (31 to 70), and when it is a `Van` it reads the `Van` limits (8 to 14). One line of code, different behaviour depending on the vehicle, which is a quiet example of polymorphism.

```
@property
def passengers(self):
    return tuple(self.__passengers)

@property
def available_seats(self):
    return self._capacity - len(self.__passengers)
```

The passenger's property hands out a **tuple copy** of the list. A tuple cannot be changed, so outsiders can look at who is on board but cannot add or remove anyone. `available_seats` is calculated fresh each time it is asked (capacity minus passengers), so it can never disagree with the real passenger list.

***Transport analogy.** The passengers property is a glass window into the vehicle: you can see who is inside but you cannot open the door. The available_seats property is the dashboard seat counter, which counts again each time you look.*

The conductor: `assign_route`, `board` and `alight`

```
def board(self, learner):
    if self.available_seats == 0:
        raise VehicleFullError(f"{self.TYPE_NAME} {self._registration} is full ...")
    self.__passengers.append(learner)

def alight(self, learner):
    self.__passengers.remove(learner)
```

`board` is where the rule "prevent allocation when the vehicle is full" lives. The vehicle guards its own capacity, so it is impossible to overfill it no matter which part of the program asks. If seats remain, the learner is added to the private list. `alight` removes a learner who is getting off. In this lean version `alight` trusts the system to only remove people who really are on board, and the system checks that through the subscription first. `assign_route` is similar: it refuses if the vehicle already serves a route, so a vehicle can never be posted to two routes.

The two abstract methods

```
@abstractmethod
def calculate_operating_cost(self, distance_km):
    """Monthly cost of running the vehicle on a route of this one-way length."""

@abstractmethod
def transport_charge(self, distance_km):
    """Monthly fee for ONE learner on a route of this one-way length."""
```

These are the **abstract methods** the assignment demands. They have no working code, only a description. Any subclass that forgets to write them cannot be created at all. The parent is saying "every vehicle must be able to work out its own running cost and its own fee, but I cannot know how, because that depends on the kind of vehicle".

***Transport analogy.** The parent class is a blank rate card issued to every vehicle. Each vehicle type must fill in its own numbers before it is allowed on the road.*

6. Bus, Minibus and Van: The Vehicle Family

```
class Bus(TransportVehicle):
    TYPE_NAME, MIN_CAPACITY, MAX_CAPACITY = "Bus", 31, 70

    def calculate_operating_cost(self, distance_km):
        return round(distance_km * 2 * self.SCHOOL_DAYS * 3500 + 600_000 + 250_000)

    def transport_charge(self, distance_km):
        return round(60_000 + 1_500 * distance_km)
```

Inheritance

`class Bus(TransportVehicle)` says "a Bus is a kind of `TransportVehicle`". It has no constructor, no `board()`, no seat counting, and yet it can do all of them, because it **inherits** everything from its parent.

Only what is different is written here: the name and seat limits (replacing the parent's defaults) and the two cost formulas. Minibus and Van are built the same way.

Overriding and polymorphism

Every subclass **overrides** the two abstract methods with its own formula. The cost formula reads, in plain words: distance, times 2 for the journey there and back, times 22 school days, times the fuel cost per kilometre, plus the staff cost and the maintenance cost. The fee formula is a fixed base charge plus an amount per kilometre.

Type	Seats	Fuel per km	Staff	Maintenance	Fee per learner
Bus	31 to 70	3,500	600,000 (driver and conductor)	250,000	60,000 + 1,500 per km
Minibus	15 to 30	2,200	400,000 (driver)	150,000	80,000 + 2,000 per km
Van	8 to 14	1,400	300,000 (driver)	100,000	100,000 + 2,500 per km

All amounts are sample figures in Uganda shillings. Worked example for a 10 km route: a Bus charges $60,000 + 15,000 = 75,000$ per learner, a Minibus charges $80,000 + 20,000 = 100,000$ and a Van charges $100,000 + 25,000 = 125,000$. Smaller vehicles charge more per learner because fewer people share the cost.

Polymorphism is what happens when the program uses these methods. In the report, the code loops over a mixed list of vehicles and calls the same method on each:

```
for v in self.s.vehicles:
    print(... ugx(v.transport_charge(10)))
```

The loop never asks "are you a bus or a van?". It just says `transport_charge(10)` and each object answers with its own formula. The Subscription class relies on the same trick to fix a learner's price.

Transport analogy. The accountant walks through the yard and asks every vehicle the same question: "what is your monthly fee for a 10 km route?" The bus, minibus and van each answer with their own figure, and the accountant does not need to know which is which.

7. The Route Class: The Road Plan

```
def __init__(self, route_id, name, stops, distance_km):
    self._route_id = route_id
    self._vehicles = []
    name = " ".join(str(name).split())
    stops = [s.strip() for s in stops if s.strip()]
    if len(name) < 3:
        raise ValueError("Route name must be at least 3 characters.")
    if len(stops) < 2:
        raise ValueError("A route needs at least 2 stops.")
    self.name, self.stops = name, stops
    self.distance_km = distance_km
```

A route has a protected ID, a protected list of vehicles, a public name and a public list of stops. The constructor cleans the name, removes empty stops (the list comprehension keeps only stops that still have text after trimming), and rejects a route with a short name or fewer than two stops. Its distance goes through a property with a setter, so it must be more than 0 and at most 200 km.

```
def add_vehicle(self, vehicle):
```

```

        vehicle.assign_route(self)
        self._vehicles.append(vehicle)

def seats_used_and_total(self):
    return (sum(len(v.passengers) for v in self._vehicles), sum(v.capacity for v in self._vehicles))

```

`add_vehicle` shows objects **collaborating**. The route does not simply append the vehicle. It first asks the vehicle to accept the route through `assign_route`, and that call raises an error if the vehicle already has one. Only if it succeeds does the route store the vehicle. `seats_used_and_total` adds up the passengers and seats of all its vehicles and returns two numbers together.

This is an **aggregation**. A route holds vehicles, but the vehicles exist independently: they were created earlier and are simply posted to the route.

***Transport analogy.** The route is a road plan pinned to the notice board. Vehicles are posted to it, but if the plan were torn down the vehicles would still exist in the yard.*

8. The Subscription Class: The Seat Booking Ticket

```

class Subscription:
    def __init__(self, sub_id, learner, route, vehicle):
        self.sub_id = sub_id
        self.learner, self.route, self.vehicle = learner, route, vehicle
        self._monthly_charge = vehicle.transport_charge(route.distance_km)
        self.active = True

```

A subscription joins three objects together: the learner, the route and the vehicle. It is an **association class**, a class whose purpose is to record the link between other objects along with extra facts about that link, here the price and whether it is still active. The line `self.learner, self.route, self.vehicle = learner, route, vehicle` assigns all three at once.

The price is fixed at the moment of creation by asking the vehicle for its fee on this route's distance. This is a **dependency**: the subscription uses the vehicle's method but does not need to know what type of vehicle it is, which is polymorphism at work again. The charge is protected, with a read-only property so nobody can quietly change a price afterwards. The active flag lets the system cancel a subscription without deleting it, which keeps a history.

***Transport analogy.** The subscription is the booking ticket handed to a learner: name, route, vehicle and the price printed on it when it was issued. If fees change next term, tickets already issued do not change. When a learner leaves, the ticket is stamped CANCELLED, not thrown away.*

9. The TransportSystem Class: The Transport Office

This class is the manager. It owns all the registers and carries out every transaction, so the menu does not have to. Making objects work together instead of putting all the logic in the menu is requirement 11.

```

def __init__(self, school_name="Sunrise Junior School"):
    self.school_name = school_name
    self._learners, self._vehicles, self._routes = {}, {}, {}
    self._subscriptions = []

```

The three curly-bracket pairs are **dictionaries**: registers where each entry is found by its key. Learners are looked up by their ID, vehicles by registration, routes by route ID. Subscriptions live in a plain list. Because the system creates and owns all of these, this relationship is **composition**: they belong to the system and are created inside it.

The read-only views and the finder

```
learners = property(lambda self: list(self._learners.values()))

@staticmethod
def _find(store, key, label):
    key = str(key).strip().upper()
    if key not in store:
        raise TransportError(f"No {label} found with '{key}'")
    return store[key]
```

The first line is a compact way of writing a read-only `@property`: it hands out a fresh list of all learners without exposing the register itself. `_find` is the one place that searches a register. It tidies the key, and if the key is missing it raises a clear error, otherwise returns the item. `get_learner`, `get_vehicle` and `get_route` are one-line wrappers around it, so the searching code is written once instead of three times. `@staticmethod` means it does not need `self`.

Transport analogy. `_find` is the records clerk. Hand him a register and a number, and he either returns the file or calls out "no such learner".

Registering things

```
def register_learner(self, name, grade, contact):
    learner = Learner(f"L{len(self._learners) + 1:03d}", name, grade, contact)
    self._learners[learner.learner_id] = learner
    return learner
```

The ID is built from the number of learners so far plus one, formatted with three digits, so the first learner is L001, then L002, and so on. The new learner object is created (and validated by its own constructor) and filed in the register. Routes work the same way with IDs like R01.

```
VEHICLE_TYPES = {"bus": Bus, "minibus": Minibus, "van": Van}

def add_vehicle(self, vehicle_type, registration, capacity):
    cls = self.VEHICLE_TYPES.get(str(vehicle_type).strip().lower())
    if cls is None:
        raise TransportError("Vehicle type must be bus, minibus or van.")
    vehicle = cls(registration, capacity)
    ...
```

The dictionary maps the words typed by the user to the actual classes. Looking up "van" returns the class `Van` itself, and `cls(registration, capacity)` then builds a vehicle of that type. That single line creates a `Bus`, `Minibus` or `Van` depending on the word, without a chain of if-statements. The method then rejects a duplicate registration before filing the vehicle.

Transport analogy. The dictionary is the vehicle showroom catalogue. Say the word "van" and the office orders a van from the van factory; say "bus" and it orders a bus.

Assigning a learner: subscribe

```
def subscribe(self, learner_id, route_id, registration):
    learner, route, vehicle = self.get_learner(learner_id), self.get_route(route_id), self.get_vehicle(regi
    if self.active_subscription(learner.learner_id):
        raise TransportError(f"{learner.name} is already assigned to a vehicle. ...")
    if vehicle.route is not route:
        raise TransportError(f"Vehicle {vehicle.registration} does not serve route '{route.name}'.")
    vehicle.board(learner)
    sub = Subscription(f"S{len(self._subscriptions) + 1:03d}", learner, route, vehicle)
    self._subscriptions.append(sub)
    return sub
```

This is the busiest transaction and it works like the checks before a bus leaves. First the three items are found (each raises an error if unknown). Second, the learner must not already have an active subscription. Third, the chosen vehicle must actually serve the chosen route. Fourth, the vehicle is asked to board the learner, which raises the full-vehicle error if there is no seat. Only when all four checks pass is a subscription created and recorded. The order matters: because board comes before the subscription is created, a failure leaves nothing half-done.

The helper `active_subscription` uses `next(...)` over a generator expression. In plain words, it walks along the subscription list and stops at the first active ticket belonging to that learner, or returns `None` if there is none.

```
def unsubscribe(self, learner_id):
    learner = self.get_learner(learner_id)
    sub = self.active_subscription(learner.learner_id)
    if sub is None:
        raise TransportError(f"{learner.name} is not using school transport.")
    sub.vehicle.alight(learner)
    sub.active = False
    return sub
```

Removing a learner is the reverse. Find the active ticket, complain if there is none, ask the vehicle to let the learner alight (freeing a seat) and mark the ticket inactive.

Searching and the monthly position

`search_learners` accepts an ID or part of a name (case ignored) and returns every match.

`monthly_position` builds the money report: for every vehicle that serves a route it adds up the fees of the active subscriptions on that vehicle (its income) and asks the vehicle for its operating cost through the overridden method, then returns income and cost side by side.

Transport analogy. *The TransportSystem is the transport manager's office. Registers on the shelves, a clerk who finds files, a boarding gate with four checks, and an accountant comparing income against running costs.*

10. The Sample Data

`load_sample_data` is a normal function, not a class. It uses the system's own methods to register twelve learners, create two routes, add a van (8 seats), a bus (40 seats) and a minibus, post the van and bus to routes, and subscribe ten of the learners. The van is filled to capacity and two learners (L011 and L012) are left unassigned so that, during the demonstration, you can try to add one of them to the full van and show a rejection. The minibus is deliberately left without a route so you can show that a vehicle must serve the route.

11. The Menu: The Reception Desk

The menu has one job: talk to the person at the keyboard. It contains no business rules at all.

```
def ask_text(prompt):
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print(" Input cannot be empty.")

def ask_number(prompt, kind=int):
    while True:
        try:
            return kind(input(prompt).strip())
```

```
except ValueError:
    print(" Please enter a valid number.")
```

These two helpers keep asking until the answer is acceptable. `ask_text` refuses empty input. `ask_number` tries to convert the text into a whole number or a decimal (the `kind` parameter chooses which) and asks again if that fails.

```
self.actions = {
    "1": ("Register learner", self.register_learner),
    ...
}
```

The actions dictionary is a switchboard. Each menu number maps to a label and to the method that handles it, so adding a new option is one new line instead of another block of if-statements. The `run` method prints the menu in a loop, reads the choice, and calls the matching method inside a protective block:

```
try:
    self.actions[choice][1]()
except (ValueError, TransportError) as err:
    print(f" REJECTED: {err}")
```

If anything anywhere in the program raises a `ValueError` or a `TransportError`, the error travels up to this block, the message is printed after the word `REJECTED`, and the menu carries on. This single block is what makes every invalid operation in the whole program end politely.

Each action method is short. For example, `assign_learner` asks for three values and passes them to `self.s.subscribe(...)` (here `self.s` is the `TransportSystem`), then prints the result. The menu never checks capacity, never calculates a fee and never looks inside a vehicle.

Transport analogy. The menu is the reception desk. The receptionist takes your request, hands it to the transport office, and reads out the answer or the refusal. She never decides who may board a vehicle.

The last lines of the file, `if __name__ == "__main__":`, mean "run the menu only when this file is started directly". It creates a `TransportSystem`, gives it to a `TransportMenu` and calls `run()`.

12. Following One Request Through the Whole Program

The best way to see collaboration is to follow one action from start to finish. Suppose you choose option 5 and try to assign learner L011 to route R01 on the full van UBA 101A.

Step	Who acts	What happens
1	<code>TransportMenu.run</code>	Reads "5" and calls <code>assign_learner</code> inside the try block.
2	<code>TransportMenu.assign_learner</code>	Asks for the learner ID, route ID and registration, then calls <code>system.subscribe</code> .
3	<code>TransportSystem.subscribe</code>	Finds the learner, route and vehicle. All three exist.
4	<code>TransportSystem.subscribe</code>	Checks L011 has no active subscription. Passes.
5	<code>TransportSystem.subscribe</code>	Checks that the van serves route R01. Passes.
6	<code>TransportVehicle.board</code>	Available seats is 0, so it raises <code>VehicleFullError</code> .
7	Python	The error rushes upward. No Subscription is created and no state has changed.
8	<code>TransportMenu.run</code>	The except block catches it and prints: <code>REJECTED: Van UBA 101A is full (8/8)</code> .

Four objects (the menu, the system, the vehicle and, at the very edge, the exception) each did one small part, and no single class knew everything. That is what "objects collaborating" means.

13. Answers to the Eleven Assignment Questions

Each answer below says how the code meets the requirement and where to point during the demonstration.

1. Identify the main classes and their responsibilities

The problem describes learners, vehicles, routes and subscriptions, so those became classes. `Learner` holds and validates a learner's details. `TransportVehicle` and its three children know their seats, their passengers, their route and their costs. `Route` holds a name, stops, a distance and its vehicles.

`Subscription` records one learner riding one vehicle on one route at a fixed monthly price.

`TransportSystem` owns the registers and performs the transactions, and `TransportMenu` only handles input and output. See section 1.3.

2. Constructors, attributes and methods

Every class has an `__init__` constructor, and the sample data shows objects being created. Each class has suitable attributes (for example a vehicle's registration, capacity, route and passengers) and methods (`board`, `add_vehicle`, `subscribe` and so on).

3. Encapsulation with public, protected and private attributes

All three levels appear. The private one is `__passengers` in the vehicle. Protected ones include `_route`, `_capacity`, `_learner_id` and `_monthly_charge`. Public ones include `grade`, a route's name and stops, and a subscription's active flag. Properties with validating setters protect a learner's name and contact, a vehicle's capacity and a route's distance, while `learner_id` and `registration` are read-only.

4. Meaningful object relationships

Each relationship does real work. **Aggregation**: a route holds vehicles that exist independently. A vehicle also holds its learners as passengers. **Association**: the `Subscription` class links a learner, a route and a vehicle and carries the price. **Composition**: the `TransportSystem` creates and owns the learners, vehicles, routes and subscriptions. **Dependency**: a `Subscription` uses a vehicle's `transport_charge()` to fix its price, and the menu depends on the system.

5. The UML class diagram

The UML diagram has not been drawn yet, and it is the next item on the to-do list. The brief says it should be prepared before implementation, so it must match this final code exactly: the classes above, their attributes and methods with visibility marks, the inheritance arrows from `Bus`, `Minibus` and `Van` to `TransportVehicle`, and multiplicities such as one `Route` to one or more `Vehicles` and one `Vehicle` to zero or more `Subscriptions`. This section will be updated once the diagram is ready.

6. A meaningful inheritance hierarchy

`TransportVehicle` is the superclass and `Bus`, `Minibus` and `Van` are three subclasses. They share the seat counting, boarding, route and validation code from the parent and differ only in their limits and formulas. A second small hierarchy is `VehicleFullError` under `TransportError`.

7. Method overriding and polymorphism

Each subclass overrides `calculate_operating_cost` and `transport_charge`. Menu option 10 loops over a mixed list of vehicles calling the same method and gets different results, and `Subscription` and `monthly_position` call these methods without ever checking the vehicle's type.

8. Abstraction

`TransportVehicle(ABC)` is an abstract base class with two `@abstractmethod` declarations that every subclass must implement. Trying to create a `TransportVehicle` directly, or a subclass that omits either method, fails.

9. A menu for registering, transacting, searching and reporting

Options 1 to 3 register learners, vehicles and routes. Options 4 to 6 are the main transactions: post a vehicle to a route, assign a learner and remove a learner. Options 7 to 9 display passengers, search for a learner and show vehicles and routes. Options 10 and 11 are the reports (charges and costs, and occupancy). Option 12 loads sample data for the demonstration.

10. Validation and clear messages

Validation happens at three layers. Setters and constructors reject malformed data with `ValueError`. The system and vehicles reject rule violations (duplicates, a full vehicle, a learner already assigned, a vehicle on the wrong route) with `TransportError` and `VehicleFullError`. The menu re-asks for empty or non-numeric input and prints every rejection as a clear `REJECTED` message.

11. Objects collaborating rather than one large class

The menu only asks and prints. The system coordinates. The vehicle guards its own capacity, the route asks the vehicle to accept it, the subscription asks the vehicle for its price, and the learner validates itself. Section 12 traces one request through these objects.

14. Quick Guide for the Demonstration

During the demonstration the group must show sample records, the major transactions, at least one rejected operation, and point to where each principle lives in the code.

Principle	Where to point	What to say
Encapsulation	<code>TransportVehicle: __passengers</code> , the capacity setter; <code>Learner: name</code> setter	Private list, controlled access, validation in setters.
Object relationships	<code>Route.add_vehicle</code> ; <code>Subscription</code> ; <code>TransportSystem.__init__</code>	Aggregation, association class, composition.
Inheritance	<code>class Bus(TransportVehicle)</code> and its two siblings	Children reuse the parent and change only limits and formulas.
Polymorphism	<code>charges_report</code> loop; <code>Subscription.__init__</code>	Same method call, different results per vehicle.
Abstraction	<code>TransportVehicle(ABC)</code> and <code>@abstractmethod</code>	Cannot create a plain vehicle; children must implement both methods.
Rejected operation	Menu 12, then 5 with L011, R01, UBA 101A	Van is full, so the vehicle raises <code>VehicleFullError</code> .
Another rejection	Menu 5 with L001 (already assigned), or 1 with name "J"	Duplicate assignment, or validation in the name setter.

Glossary

Term	Simple meaning	Transport picture
Class	A blueprint for making objects	The design for a van model
Object	One real thing built from a class	The actual van in the yard
Constructor	Code that sets up a new object	Filling in the papers when a vehicle joins the fleet
self	The object currently being worked on	This particular vehicle
Attribute	A piece of data stored in an object	A vehicle's number plate
Method	An action an object can perform	Board, alight, work out the fee
Property / setter	Controlled read and write access	Glass window and gate guard
Encapsulation	Hiding data behind controlled access	Locked passenger compartment
Inheritance	A child class reuses its parent	All vehicles share basic vehicle features
Overriding	A child replaces a parent's method	Each vehicle fills in its own rate card
Polymorphism	Same call, different behaviour	Ask any vehicle for its fee
Abstract class	A blueprint that cannot be instantiated	The idea of a vehicle, never parked
Exception	A signal that something went wrong	A red flag from the conductor
Aggregation	Holds parts that exist independently	Route and its vehicles
Composition	Owns parts that live inside it	Office and its registers
Association	A link between objects, possibly with details	Booking ticket
Dependency	Uses another class temporarily	Ticket asks a vehicle for its price